



Basic Spectral Analysis 2

基本的なスペクトル分析2

SILAA, Victor Alex
釧路工業高等専門学校
創造工学科(情報分野)
silaa@kushiro.kosen-ac.jp

離散フーリエ変換対 (**DFT** および **IDFT**) の実装・利用の方法について理解しよう.

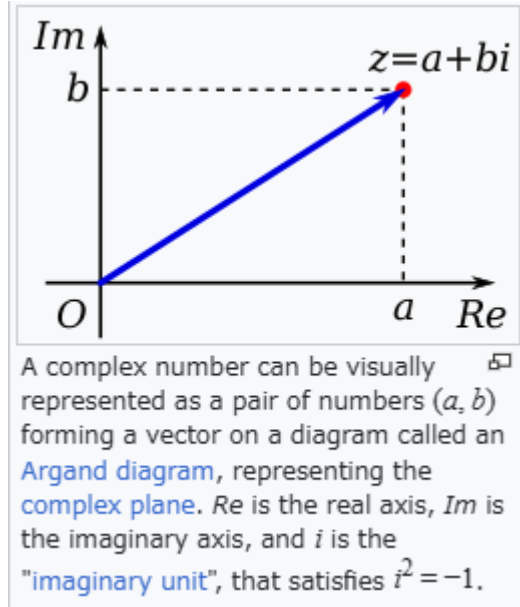
Understand how to implement and use discrete Fourier transform pairs (DFT and IDFT).

教科書の該当範囲: 第4章

Textbook Coverage: Chapter 4

C言語における複素数の利用方法

How to use complex numbers in C language



DFT および IDFT の計算には，複素数が利用されるので，C言語プログラミングの知識をアップデートしておこう。

Since complex numbers are used in the DFT and IDFT calculations , update your knowledge of C programming

C89 の複素数 complex numbers in C89

C89（ANSI C）等の古い標準 C では、複素数用のデータ型や関数を各プログラマが独自に実装していた。

In older C standards such as C89 (ANSI C), programmers implemented their own data types and functions for complex numbers.

複素数型の定義例

Example of complex number types definitions

//Complex data type

//Numeric setting function in Cartesian coordinate format

//Numeric setting function in Polar coordinate format

// Additive Function

```
#include <math.h>

// 複素数のデータ型
typedef struct { double re, im; } Complex;

// 直交座標形式の数値設定関数
// 数学:  $z = x + iy$       コード:  $z = \text{cValue}(x, y)$ ;
Complex cValue(double x, double y)
{
    Complex z;
    z.re = x;
    z.im = y;
    return (z);
}

// 極座標形式の数値設定関数
// 数学:  $z = r \exp(ja)$       コード:  $z = \text{cPolar}(r, a)$ ;
Complex cPolar(double r, double a)
{
    return (cValue(r*cos(a), r*sin(a)));
}

// 加算関数
// 数学:  $c = a + b$       コード:  $c = \text{cAdd}(a, b)$ ;
Complex cAdd(Complex a, Complex b)
{
    return (cValue(a.re + b.re, a.im + b.im));
}
...
```

C99 の複素数 complex numbers in C99 組込機能

Built-in Function

C99 標準（や GCC）では、実数の拡張版として、複素数を取り扱えるようになった。もちろん、複素数同士の四則演算や変数への代入も可能である

C99 standard (and GCC) can handle complex numbers as an extension of real numbers. Of course, it is possible to perform four arithmetic operations on complex numbers and assign them to Variables.

使用例

Usage Example

標準ライブラリ

Standard Library

// Cartesian coordinate format

// real part

// imaginary part

// Polar coordinate form

// Absolute

// Declination

// Complex conjugate

名称	コード書式	コード例	補足
複素数型	double _Complex	double _Complex z;	複素変数 z の宣言
虚数接尾辞	i	z = 1.0 + 2.0i;	変数 z への複素数 $1 + i2$ の代入

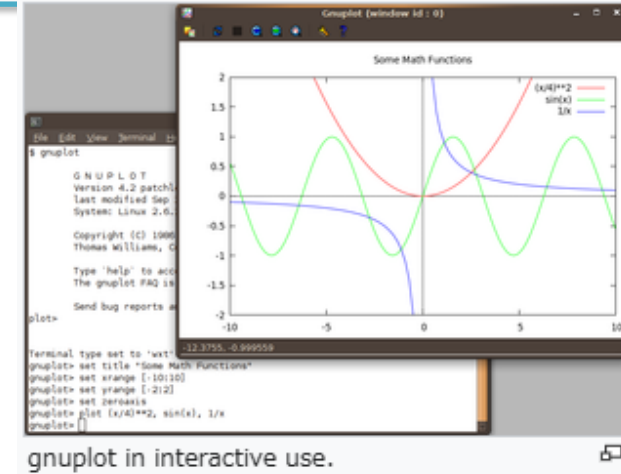
名称	数学的表記例	コード例
直交座標形式	$\dot{z} = x + iy$	z = x + I*y;
実部	$x = \text{Re } \dot{z}$	x = creal(z);
虚部	$y = \text{Im } \dot{z}$	y = cimag(z);
極座標形式	$\dot{z} = r \exp ja$	z = r*cexp(I*a);
絶対値	$r = \dot{z} $	r = cabs(z);
偏角	$a = \arg \dot{z}$	a = carg(z);
複素共役	$\dot{z}_2 = \overline{\dot{z}_1}$	z2 = conj(z1);

Google検索→ gnuplot

Google search : gnuplot

```
set title "Some Math Functions"
set xrange [-10:10]
set yrange [-2:2]
set zeroaxis
plot (x/4)**2, sin(x), 1/x
```

ref: Wikipedia



実行結果には大量の数値が含まれるので、グラフ化ツールが必須。
gnuplot（にゅーぷろっと）は、その定番。

Since the execution results contain a large number of numbers, a **graphing tool** is a must. **Gnuplot** is the **standard**.

今回の実習では、用意されているスクリプトを使うだけですが、
基本的な記述方法を理解しておくとい良いでしょう。

In this exercise we will only use the scripts provided, but it is good idea to understand how to write a basic description of gnuplot.

作業用ディレクトリ：以前の通り `~/sp/pcm/` 等

Working directory –

ソースファイル：**dft.c**

Source file – This is a working template. Data input/output and performance evaluation functions are implemented.
Analysis and Synthesis (DFT/IDFT) functions are not implemented.

ヘッダファイル：**cx.h**

Head file

gnuplot スクリプト gnuplot script

波形表示用：**dw.plot** For waveform display

スペクトル分布用：**ds.plot** For spectral distribution

これらのスクリプトを実験室以外の環境で利用するには、まず、**gnuplot** をインストールしておきましょう。

To use scripts in a non-laboratory environment, **first install gnuplot**.

動作テスト用WAVファイル例：**sin-A.wav**, **sqr-A.wav**, 等。

WAV file for operational test: sin-A.wav, sqr-A.wav

以前の基本波形生成アプリ **wave** 等で作成しておこう。

These files should be created using previous basic waveform generation application wave.



コンパイル・実行許可 Compile and execution Permission

```
$ cc dft.c -std=c99 -L. -lpcm -lm -o dft
$ chmod +x *.plot
```

実行 Execution

```
$ ./dft ファイル.wav 480 0 # 0番から480個分の標本を分析へ
...
$ ls *.txt
wf.txt # 分析対象の入力波形のデータファイル
$ ./dw.plot wf.txt & # グラフ表示
```



サンプルソース `dft.c` を元にして， 分析および合成のための関数 `DFT()` および `IDFT()` を実装せよ.

Based on the **sample source** `dft.c`, implement the **functions** `DFT()` and `IDFT()` for analysis and synthesis.

計算方法は，前回のページに示した通り， 実に簡単です． 各関数内のコードは， ほぼ， 二重ループひとつずつだけでしょう．

The code within each function should be approximately one doubled-loop each.



DFT の実装と利用

DFT implementation and use

```
$ vim dft.c
...
$ cc ...
$ ./dft ...
...

$ ls *.txt
dft.txt      # DFTによる分析結果のスペクトル分布
idft.txt     # DFT結果に対するIDFTによる合成結果の波形（入力と同じハズ）
wf.txt       # 分析対象の入力波形

$ ./ds.plot dft.txt & # スペクトル分布のグラフ表示
$ ./dw.plot idft.txt & # 合成波形のグラフ表示
$ ./dw.plot wf.txt &  # 入力波形のグラフ表示
```

spectral distribution of analysis results by DFT
waveform of synthesis result by IDFT for DFT result(same as input)
input waveform to be analyzed
graphical display of spectral distribution
graph the composite waveform
graph the input waveform

スペクトル分布および波形のグラフを観察し,実行結果の正当性を確認せよ.

Check the validity of the results by observing the spectral distribution and the graph of the waveforms

例えば、入力データが正弦波の A 音（ラ）なら、スペクトルのピークが 440 Hz 付近にあるか？他の入力データに対しては、どうか？

For example, if the input data is a sinusoidal A tone (la), is the spectral peak around 440 Hz? How about for other input data?

周波数領域の上半分側に折り返し成分もあるはず。

There should be a folded component in the upper half of the frequency domain.

合成波形と入力波形とは等しいか？

Is the composite waveform equal to the input waveform?

分析条件（標本数）を様々に変更し、処理時間および周波数分解能について、傾向を調査せよ。

Investigate trends in processing time and frequency resolution by varying the analysis conditions (number of samples).



処理時間については、窓幅（分析対象の収録時間）と比較しよう．

For processing time, compare with the window width (recording time of the analysis target).

周波数分解能については、スペクトルのピークが鋭いほど、分析精度が高い、ってことですよ． 例えば、和音（ドミソ）の各構成音（ド、ミ、ソ）を弁別できそうか？

For frequency resolution, the sharper the spectral peak, the more accurate the analysis. For example, can you discriminate each component (do, mi, so) of a chord (do-mi-so)?

窓関数による分解能改善.

Resolution improvement by window function.

分解能の劣化は， 入力信号の非周期性（窓幅と信号周期の不一致， 窓の両端間の不連続）が原因.

Degradation of resolution is caused by aperiodicity of the input signal (mismatch between window width and signal period, discontinuity between the two ends of the window).

分析の前処理として， 窓関数（周期＝窓幅な周期関数）によって信号を振幅変調しておく

1. As a **pre-processing step**, the signal is amplitude-modulated by a window function (a periodic function where period = window width).
2. To deal with **Spectral Leakage** when **unfinished wave** pass through FFT.



代表例：ハニング窓. Hanning window function

Hanning window function.

$$x_n \rightarrow w_n x_n$$

$$w_n = \frac{1}{2} \left(1 - \cos \frac{2\pi n}{N} \right) = \sin^2 \frac{\pi n}{N}$$

他のは,

- Tukey
- Hamming
- Etc . . .

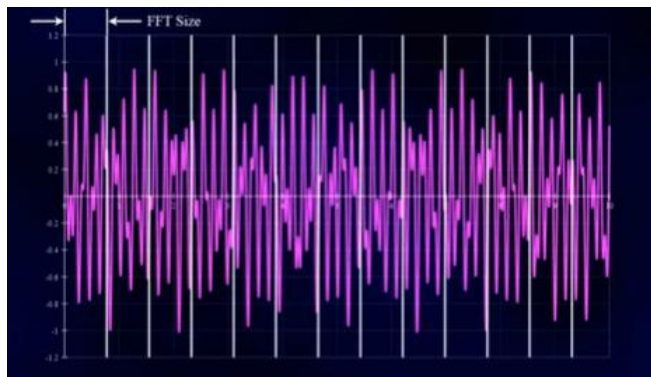
関数 **DFT()** および **IDFT()** 内の反復計算に漸化式を適用し，計算コストを削減

Applies an iterative formula to iterations in the functions DFT() and IDFT() to reduce computational cost

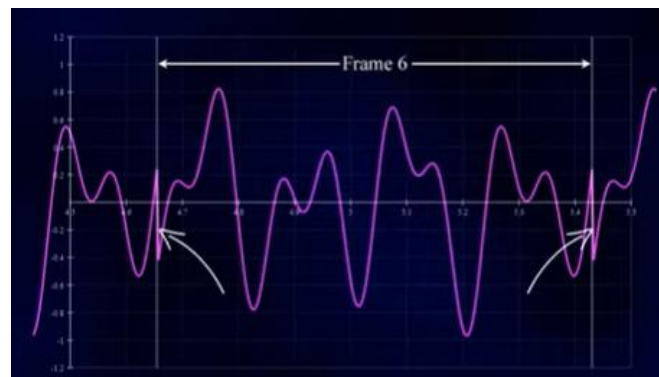
コードは複雑化しますが，処理時間は大幅に短縮可能ですよ．ただし，アルゴリズム的には同じなので， $O(N^2)$ は変わりません．

Although the code is more complex, the processing time can be significantly reduced. However, since the algorithm is the same, $O(N^2)$ remains the same.

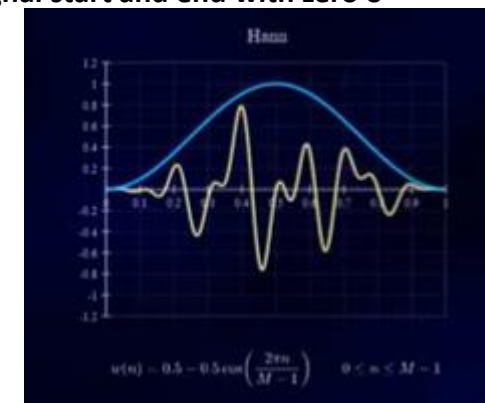
FFT-size = Power of 2 but, (signal longer than size of FFT) → cut into small FFT size – Frames To avoid cut of signal → Window function ensure signal start and end with zero 0



→



→



DFT の実装(と高性能化)についての取組成果を報告せよ.

Report the results of your efforts to implement (and improve the performance of) DFT

1. 件名 → **sp-1805**
2. 提出方法 → **Teams** 宛先：**シラー ヴィクター アレックス_釧路**
3. 提出期限 → **6月1日（月） 17:00hrs**
4. 提出内容 → （本文）
 - 学年・分野，出席番号，氏名
 - 作成した **dft.c** のソースコード
 - （工夫点および，それによる性能向上への効果の説明）
 - **DFT・IDFT**に対する感想・考察